Backend Application using Express - Marking Rubric (50 marks)

Design Phase (10 marks)

Entity Relationship Diagram (5 marks)

Band	Marks	Criteria
A	4-5	ERD is professionally designed with all six models clearly represented. All fields include accurate data types and comprehensive constraints. Relationships are correctly identified and labeled with appropriate cardinality. Enum values are fully documented. Uses proper ERD notation consistently.
B	3.2-3.9	ERD shows all six models with most fields including data types and constraints. Relationships are mostly correct with minor notation inconsistencies. Enum values are documented. Generally follows ERD conventions with minor errors.
C	2.5-3.1	ERD includes six models but some fields are missing data types or constraints. Relationships are present but may have some errors in cardinality. Enum values may be incomplete. Some notation inconsistencies.
D/E	0-2.4	ERD is incomplete or poorly designed. Fewer than six models, missing field information, incorrect or missing relationships, or does not follow ERD conventions. Difficult to understand or implement from.

API Endpoints (5 marks)

Band	Marks	Criteria
A	4-5	Complete and comprehensive endpoint documentation in clear table format. All required information present for every endpoint: HTTP method, URL, description, authentication requirements, RBAC requirements and all parameter types. Follows RESTful conventions. Well-organised and professional.
B	3.2-3.9	Good endpoint documentation with most information present. Minor omissions in parameter information or RBAC requirements. Generally follows RESTful conventions with minor inconsistencies. Clear and usable.
C	2.5-3.1	Basic endpoint documentation present but missing some information. Some endpoints lack complete parameter information or RBAC requirements. May not fully follow RESTful conventions. Adequate but could be clearer.
D/E	0-2.4	Incomplete endpoint documentation. Missing significant endpoint information or unclear descriptions. Does not follow required table format well.

Development Phase (25 marks)

Project Management (2 marks)

Band	Marks	Criteria
A	1.6-2	Excellent use of GitHub Project with clear sprint planning and task breakdown. Issues are well-defined with appropriate labels and assignments. Tasks move systematically through Backlog, In Progress and Done columns. Demonstrates strong agile methodology implementation.
B	1.3-1.5	Good project management with GitHub Project set up correctly. Most tasks are defined as issues and tracked through columns. Sprint planning is present but may lack some detail. Generally follows agile methodology.
C	1-1.2	Basic GitHub Project implementation. Some tasks tracked as issues but inconsistent use of columns. Sprint planning may be minimal or unclear. Shows some understanding of agile methodology.
D/E	0-0.9	Poor project management. GitHub Project not properly set up or not used consistently. Tasks not tracked as issues or sprints not defined. Little evidence of agile methodology.

Database (1 mark)

Band	Marks	Criteria
A	0.8-1	Prisma is correctly configured and used across all environments. Schema is well-designed with proper types and relations. Migrations are properly managed.
B	0.7	Prisma is used in all required environments with mostly correct configuration. Schema is functional with minor issues. Migrations present.
C	0.5-0.6	Prisma is implemented but may have configuration issues in some environments. Schema works but may have design flaws. Migration management inconsistent.
D/E	0-0.4	Prisma has significant configuration issues or not properly implemented. Schema has major problems or database functionality is broken.

Models (3 marks)

Band	Marks	Criteria
A	2.4-3	Six models implemented with all requirements met. Each model has minimum four fields. Two enum fields properly implemented with appropriate values. All four relationship types correctly implemented. Models are well-designed and follow best practices.
B	2-2.3	Six models implemented with most requirements met. Minor issues with field counts or relationships. Enum fields present and functional. Relationships mostly correct with minor implementation issues.

Band	Marks	Criteria
C	1.5-1.9	Six models present but some do not meet field requirements. Enum fields may be incomplete. Some relationships missing or incorrectly implemented. Models are functional but have design issues.
D/E	0-1.4	Models have significant issues. Fewer than six models, missing fields, incorrect relationships, or poor design. Functionality compromised.

CRUD (10 marks)

Endpoints Implementation (4 marks)

Band	Marks	Criteria
A	3.2-4	All CRUD operations fully implemented for all six models. Authentication endpoints work correctly with token-based authentication and complex features including account lockout and password confirmation. Health check endpoint provides comprehensive status including database connectivity and uptime. Catch-all endpoint properly handles undefined routes. All endpoints follow RESTful conventions and handle edge cases.
B	2.6-3.1	CRUD operations implemented for all models with minor issues. Authentication endpoints functional. Health check provides basic status. Catch-all endpoint present. Generally follows RESTful conventions with some inconsistencies.
C	2-2.5	CRUD operations present but some incomplete or buggy. Authentication endpoints work but may have issues. Health check basic or missing some checks. Catch-all endpoint present but may not handle all cases. Some deviation from RESTful conventions.
D/E	0-1.9	CRUD operations have significant problems across models. Authentication endpoints have major issues or missing. Health check inadequate or missing. Poor endpoint design.

Validation (1 mark)

Band	Marks	Criteria
A	0.8-1	Comprehensive validation on all create and update operations. Validates data types, required fields, constraints and business rules. Returns clear, specific error messages. Properly sanitises input.
B	0.7	Good validation covering most fields in create and update operations. Returns meaningful error messages. Some edge cases may not be covered.
C	0.5-0.6	Basic validation present but incomplete. May miss some fields or edge cases. Error messages present but could be more specific.
D/E	0-0.4	Minimal or no validation. Operations accept invalid data. Poor or missing error messages. No input sanitisation.

Filtering, Sorting and Pagination (1 mark)

Band	Marks	Criteria
A	0.8-1	Complex filtering, sorting and pagination fully implemented on all read all operations. Multiple filter criteria supported. Flexible sorting options. Proper pagination with page size controls. Query parameters well-designed.
B	0.7	Filtering, sorting and pagination implemented on all models with minor limitations. Basic filter and sort options work correctly. Pagination functional.
C	0.5-0.6	Basic filtering, sorting, or pagination present but incomplete across models. Some features may be missing or inconsistent. Limited options available.
D/E	0-0.4	Filtering, sorting and pagination largely missing or completely broken. Does not work across models.

Role-Based Access Control (1 mark)

Band	Marks	Criteria
A	0.8-1	RBAC fully implemented with at least two distinct roles having clearly different permissions. Permissions properly enforced across all relevant endpoints. Unauthorised access correctly denied with appropriate status codes. Role assignment and checking robust.
B	0.7	RBAC implemented with two roles having different permissions. Most endpoints properly protected. Minor issues with permission enforcement or edge cases.
C	0.5-0.6	Basic RBAC present with two roles but permissions may not be significantly different. Some endpoints not properly protected. Inconsistent enforcement.
D/E	0-0.4	RBAC missing, incomplete, or broken. Roles not properly differentiated. Permissions not enforced. Security vulnerabilities present.

Content Negotiation Middleware (1 mark)

Band	Marks	Criteria
A	0.8-1	Content negotiation middleware properly implemented and applied to all endpoints. Consistently returns responses in JSON format with correct Content-Type headers. Handles Accept headers appropriately.
B	0.7	Content negotiation middleware implemented and used on most endpoints. Returns JSON responses with correct headers in most cases. Minor inconsistencies.
C	0.5-0.6	Basic content negotiation present but inconsistent application across endpoints. JSON responses mostly correct but may have header issues.
D/E	0-0.4	Content negotiation missing or improperly implemented. Inconsistent response formats. Incorrect or missing headers.

Cache Middleware (1 mark)

Band	Marks	Criteria
A	0.8-1	Cache middleware correctly implemented for all read all and read by ID operations across all models. Proper cache headers set. Cache invalidation handled appropriately on updates/deletes. Improves performance measurably.
B	0.7	Cache middleware implemented for most read operations. Cache headers generally correct. Basic cache invalidation present. Works but may have minor issues.
C	0.5-0.6	Basic cache implementation present but incomplete coverage across models or operation types. Cache headers may be incorrect. Invalidation may not work properly.
D/E	0-0.4	Cache middleware missing, broken, or incorrectly implemented. Does not improve performance or causes issues.

Rate Limiting Middleware (1 mark)

Band	Marks	Criteria
A	0.8-1	Rate limiting middleware properly implemented with different limits based on user roles. Correctly identifies and limits requests per role. Returns appropriate 429 status codes when limits exceeded. Configuration is sensible and well-documented.
B	0.7	Rate limiting implemented with role-based differentiation. Works correctly in most cases. May have minor issues with edge cases or configuration.
C	0.5-0.6	Basic rate limiting present with some role differentiation but may not work consistently. Limits may not be appropriate or well-configured.
D/E	0-0.4	Rate limiting missing, broken, or not differentiated by role. Does not effectively prevent abuse.

API Tests (4 marks)

Band	Marks	Criteria
A	3.2-4	Comprehensive test suite covering all requirements: CRUD operations for all models, authentication endpoints, health check, catch-all, validation, filtering/sorting/pagination and role-based permissions. Tests run successfully against both development and production environments. Well-organised with clear test descriptions. High code coverage. Tests are maintainable and follow best practices.
B	2.6-3.1	Good test coverage of most requirements. Tests work in both environments with possible minor issues. Most critical paths tested. Tests are organised and mostly follow best practices. Some edge cases may not be covered.
C	2-2.5	Basic test coverage present but incomplete. Some required areas not tested or tests only work in one environment. Tests may be poorly organised or have reliability issues. Limited edge case coverage.

Band	Marks	Criteria
D/E	0-1.9	Minimal or no test coverage. Many required areas not tested. Tests have significant problems, are unreliable, or missing entirely.

Scripts (2 marks)

Band	Marks	Criteria
A	1.6-2	All required scripts present and functional: dev, format, lint, database setup, migrations, reset, seed, build and test scripts. Scripts are well-named, documented and work reliably. Error handling included where appropriate.
B	1.3-1.5	Most scripts present and functional. Minor issues with some scripts or documentation. All critical scripts work correctly.
C	1-1.2	Basic scripts present but some missing or not working correctly. May have issues with database scripts or seeding. Documentation minimal.
D/E	0-0.9	Many scripts missing or broken. Cannot reliably run application or tests using provided scripts.

Deployment (3 marks)

Band	Marks	Criteria
A	2.4-3	Application successfully deployed to Render and fully functional. All endpoints work correctly in production. Environment variables properly configured. Database connected and operational. Comprehensive verification performed using Postman. Production environment is stable and performant.
B	2-2.3	Application deployed and mostly functional. Most endpoints work with minor issues. Environment properly configured. Database connected. Verification performed with some test coverage.
C	1.5-1.9	Application deployed but has functional issues. Some endpoints not working or performance problems. Configuration issues present. Limited verification performed.
D/E	0-1.4	Application not properly deployed, largely non-functional in production, or major features broken. Poor configuration.

Code Quality and Best Practices (15 marks)

Code Organisation (3 marks)

Band	Marks	Criteria
A	2.4-3	Excellent project structure with clear separation of concerns. Presentation and data access layers properly separated. Code is highly modular with well-defined, reusable functions and modules. Easy to navigate and understand. Follows industry best practices.

Band	Marks	Criteria
B	2-2.3	Good project structure with separation of concerns implemented. Layers are mostly separated. Code is modular with some reusable components. Generally follows best practices with minor organisational issues.
C	1.5-1.9	Basic project structure present but separation of concerns inconsistent. Some mixing of layers. Limited modularity and code reuse. Organisation could be significantly improved.
D/E	0-1.4	Poor or no project structure. Little to no separation of concerns. Code is monolithic with little modularity. Difficult to navigate.

Code Style and Formatting (2 marks)

Band	Marks	Criteria
A	1.6-2	Code is consistently linted with ESLint and formatted with Prettier. Follows all naming conventions. All variables, functions and classes have clear, descriptive, meaningful names. No linting errors. Code is highly readable.
B	1.3-1.5	Code is linted and formatted with minor inconsistencies. Generally follows naming conventions. Most names are meaningful and descriptive. Few linting errors. Code is readable.
C	1-1.2	Basic linting and formatting present but inconsistent. Naming conventions followed sometimes. Some names are unclear or not descriptive. Multiple linting errors. Readability could be improved.
D/E	0-0.9	Little or no linting/formatting. Poor naming conventions. Many linting errors. Code is difficult to read.

Error Handling (3 marks)

Band	Marks	Criteria
A	2.4-3	Exceptional error handling throughout application. All errors caught and handled gracefully with meaningful, specific error messages. Appropriate HTTP status codes used consistently. Consistent error response format across all endpoints. Database and validation errors properly handled with user-friendly messages. No stack traces exposed to clients.
B	2-2.3	Good error handling covering most scenarios. Meaningful error messages provided. HTTP status codes mostly appropriate. Consistent error format with minor exceptions. Database and validation errors handled.
C	1.5-1.9	Basic error handling present but incomplete. Some errors not caught. Error messages present but could be more specific. HTTP status codes sometimes incorrect. Error format somewhat inconsistent.

Band	Marks	Criteria
D/E	0-1.4	Poor or no error handling. Many errors not caught or cause crashes. Incorrect or missing status codes. No consistent error format. May expose sensitive information.

Security (3 marks)

Band	Marks	Criteria
A	2.4-3	Excellent security implementation. All sensitive data stored in environment variables. Comprehensive .gitignore prevents committing sensitive files. Password hashing uses strong, secure algorithm. JWT implementation follows all security best practices. No security vulnerabilities present.
B	2-2.3	Good security practices. Environment variables used for sensitive data. .gitignore present and functional. Password hashing implemented with secure algorithm. JWT implementation generally secure with minor issues. Few security concerns.
C	1.5-1.9	Basic security present but with gaps. Some sensitive data may not be in environment variables. .gitignore incomplete. Password hashing present but may use weaker algorithm. JWT implementation has security issues. Some vulnerabilities present.
D/E	0-1.4	Poor or no security. Sensitive data exposed or hardcoded. No or inadequate .gitignore. Weak password hashing or stored in plain text. JWT implementation insecure or broken. Significant security vulnerabilities.

Version Control (2 marks)

Band	Marks	Criteria
A	1.6-2	Excellent Git history. All commits use conventional commit format correctly and consistently. Commit messages are clear, descriptive and meaningful. Regular commits with small, focused changes. No large, monolithic commits. Clean commit history easy to understand and navigate.
B	1.3-1.5	Good Git history. Most commits follow conventional format. Commit messages are generally clear and descriptive. Regular commits that are mostly focused. Few large commits. History is understandable.
C	1-1.2	Basic Git usage. Some commits follow conventional format. Commit messages vary in quality. Some large or unfocused commits. History could be clearer.
D/E	0-0.9	Poor or no Git history. Few commits follow conventional format. Messages unclear or non-descriptive. Very large commits or very inconsistent pattern. History is difficult to follow.

Documentation (2 marks)

Band	Marks	Criteria

Band	Marks	Criteria
A	1.6-2	Comprehensive, well-written backend-documentation.md file. Includes clear project description and purpose, detailed installation and setup instructions, complete environment variable configuration with examples, clear instructions for running in development, complete test running instructions and working deployment URL. Documentation is professional, well-formatted and easy to follow.
B	1.3-1.5	Good documentation covering all required areas. Instructions are clear and mostly complete. Environment variables documented. Some details may be missing but documentation is usable.
C	1-1.2	Basic documentation present but incomplete or unclear in some areas. Missing some details or instructions that would make setup difficult. Environment variables listed but not explained.
D/E	0-0.9	Poor or missing documentation. Does not cover required areas. Instructions unclear or incomplete. Difficult or impossible to set up from documentation.